



Rename `std::nontype`, and make it broadly useful

Document number:

[P3774R0](#)

Date:

2025-07-15

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

Co-authors:

Bronek Kozicki <brok@incorrekt.com>,
Tomasz Kamiński <tomaszkam@gmail.com>

GitHub Issue:

wg21.link/P3774/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/nontype-again.cow



ABSTRACT: [\[P3740R1\]](#) proposed to rename `std::nontype` to `std::constant_arg`, now that the term "non-type template parameter" no longer exists in the C++26 standard. This solution found weak consensus in LEWG, but there was no time in LWG to review the change, so it did not make it into C++26.

We now propose to rename it to `std::fn` in C++26. We also propose to make it more broadly useful in C++29 by adding further members.

§ Contents

- 1 **Introduction**
 - 1.1 Why did we need a new `std::nontype` type?
 - 1.2 Refresher on what `std::nontype` accomplishes
- 2 **Proposed solution**
 - 2.1 Impact on `std::function_ref`
 - 2.2 Making `std::nontype` useful for use in algorithms and function wrappers
 - 2.3 Making `std::nontype` useful for producing function pointers



- 2.4 Naming
- 2.5 Change of header
- 2.6 Interoperability with third-party `function_refs`
- 3 Alternatives considered
 - 3.1 Using a `std::stateless` constructor tag instead
 - 3.2 Removing `std::nontype` with no replacement, revisiting for C++29
 - 3.3 Waiting for `constexpr` function parameters
 - 3.4 Replacing `std::nontype` with `std::constant_wrapper`
- 4 Implementation experience
 - 4.1 Simple rename for C++26
 - 4.2 Possible implementation of `std::fn` for C++29
- 5 Wording
- 6 References
- 7 Appendix — Follow-up wording for C++29

§ 1. Introduction

[P2472R3] proposed additional overloads for the `std::function_ref` constructor which utilize a helper type `std::nontype`. This paper was merged into [P0792R14], which was plenary-approved at Varna 2023 for C++26.

The naming choice `std::nontype` makes very little sense following [P2841R1], which introduced concept and variable template parameters. Since those are not types either, what the standard used to call "non-type template parameter" has been renamed to "constant template parameter".

NOTE: The class template is called `std::nontype_t`, and the corresponding variable template is called `std::nontype`. This document refers to the feature as a whole as `std::nontype`.

Furthermore, the `std::constant_wrapper` from [P2781R9] was accepted into C++26, and it seems like a good replacement for `std::nontype` at first glance. At Sofia 2025, the choice to rename `std::nontype` to `std::constant_arg` instead of replacing it with `std::constant_wrapper` achieved consensus, by the narrowest of margins (see P3740 minutes):

”

ACTION: Get feedback on the decision from standard library implementers.

POLL: Forward “P3740R1: Last chance to fix `std::nontype`” selecting Option A (change from `nontype` to `constant_wrapper`; also add overloads to other function wrappers (`std::function`)) to LWG for C++26 (if possible)

SF	F	N	A	SA
0	4	0	7	8

Outcome: No consensus for change

POLL: Forward “P3740R1: Last chance to fix `std::nontype`” selecting Option A (change from `nontype` to `constant_wrapper`; DO NOT add overloads to other function wrappers (`std::function`)) to LWG for C++26 (if possible)

SF	F	N	A	SA
6	7	2	6	0

Outcome: Weak consensus in favor

POLL: Forward “P3740R1: Last chance to fix `std::nontype`” selecting Option B (rename “`std::nontype`” to “`std::constant_arg`”) to LWG for C++26 (if possible)

SF	F	N	A	SA
7	9	2	2	1

Outcome: Consensus in favor

POLL: Forward “P3740R1: Last chance to fix `std::nontype`” selecting Option A (change from `nontype` to `constant_wrapper`; DO NOT add overloads to other function wrappers (`std::function`)) instead of Option B (rename “`std::nontype`” to “`std::constant_arg`”) to LWG for C++26 (if possible)

SF	F	N	A	SA
9	4	2	4	4

Outcome: No consensus for change

However, LWG had no time to review this *extremely* last-minute change, so it did not make it into C++26.



NOTE: The reason why the last poll does not change the LEWG decision is that after the **marked decision**, renaming `std::nontype` became the status quo, and the last poll did not have enough votes to change that status quo.

§ 1.1. Why did we need a new `std::nontype` type?

An obvious question may be why the existing `std::integral_constant` cannot be used instead. This has multiple reasons:

- `std::integral_constant<class T, T v>` requires specifying the type separately.
- `std::integral_constant` has a lengthy name.
- `std::integral_constant` has a call operator which returns `v`, which produces friction with the other constructors.

§ 1.2. Refresher on what `std::nontype` accomplishes

`std::nontype` is used only within constructors of `std::function_ref`:



```
// constexpr and noexcept omitted for simplicity
template<class F>      function_ref(F*);
template<class F>      function_ref(F&&);
template<auto f>        function_ref(nontype_t<f>);
template<auto f, class U> function_ref(nontype_t<f>, U&&);
template<auto f, class T> function_ref(nontype_t<f>, cv T*);
```

Intuitively, `std::function_ref` is the C++ counterpart to the C idiom of passing `void*` and a function pointer which that `void*` is fed into, as seen in `qsort`. It is common practice to provide a null pointer to `qsort` and thus rely on a "capture-less" comparison.

With `std::nontype`, `std::function_ref` can support such use "capture-less" uses:

```
// once again, some simplifications (no noexcept, no invoke, etc.) ...
template<auto f>
function_ref(nontype_t<f>) {
    // bound-entity is similar to the void* we would have stored in C.
    // We can leave it empty since we are constructing from some global callable.
    this->bound-entity = {};
    // Note that lambdas with no captures can be converted to function pointers.
    this->thunk_ptr = [](BoundEntityType, Args&&... args) -> R {
        // Notice that we don't need to capture f.
        // This is only possible because f is a constant template parameter.
        return f(std::forward<Args&&>(args)...);
    };
}
```

Crucially, it would be impossible to create a `std::function_ref` from a function directly without this helper. At best, we could store function pointer within *bound-entity* and call that function pointer from within *bound-entity*. This would introduce entirely unnecessary overhead.

Besides overhead, we need `std::nontype` constructors to make a `std::function_ref` which has the functionality provided by a lambda, but does not reference any specific lambda. This is crucial for being able to store `std::function_ref` somewhere long-term without paying attention to the lifetime of the callable type it has been initialized with.

§ 2. Proposed solution

We propose to rename `std::nontype_t` and `std::nontype` to `std::fn_t` and `std::fn`. Such a rename is well within the scope of an NB comment, so it can be applied to C++26. Furthermore, we make the type more broadly useful for C++29 by

- making `std::fn_t` convertible to function pointers, and
- giving it a call operator, which makes it usable in function wrappers other than `std::function_ref` (with the same effect), as well as algorithms.

Expressed in code, this would look as follows:

```
 template<auto V>
struct nontype_t fn_t {
```

```
explicit nontype_t fn_t() = default;

// SINCE C++29:
// various new members ...
};
template<auto V> constexpr nontype_t<V> nontype{};
template<auto V> constexpr fn_t<V> fn{};
```



Among other reasons, keeping `std::nontype` instead of using `std::constant_wrapper` in its place was controversial because there was not enough motivation to keep both. This is mainly because `std::nontype` provides no distinct functionality; it is effectively `std::constant_wrapper` without all of the extra operator overloads. What we propose addresses this problem.

§ 2.1. Impact on `std::function_ref`

`std::function_ref` would handle `std::fn` exactly the same as `std::nontype`, whether it has this C++29 functionality or not. No C++26 code would be broken by adding those new members later. That is because there are special constructors for `std::function_ref` taking `std::nontype`, and these always win in overload resolution, whether `std::nontype` has a call operator or other members, or not.

Furthermore, `std::function_ref` needs to special-case `std::nontype` to provide the `(nontype_t<f>, U*)` and `(nontype_t<f>, U&&)` constructors. That is because only `std::nontype` provides the guarantee that its address is irrelevant to `operator()`, and that it is empty. Thanks to that guarantee, `std::function_ref` can use the *bound-entity* pointer to refer to some user-specified entity, or simply make it `nullptr`.

§ 2.2. Making `std::nontype` useful for use in algorithms and function wrappers

A common recommendation when using standard library algorithms is to avoid the use of function pointers. That is because all invocations of of say, `std::for_each` with function pointers of the same type also result in identical instantiations.

Therefore, rather than making easily inlined direct calls to lambda `operator()`s, `std::for_each</* ... */ , void(*)(<int>)>` makes indirect calls through a function pointer, and unless the algorithm is inlined and the function pointer is constant-folded, this may result in degraded performance.

The C++29 `std::fn` solves this problem by "elevating" the function pointer into the type system:

```
void f(int);
std::ranges::for_each(r, f); // bad, passing function pointer
std::ranges::for_each(r, [](int x) { return f(x); }); // OK, but clunky
std::ranges::for_each(r, std::fn<f>); // OK, and less verbose
```



NOTE: `std::constant_wrapper` is not intended to address this use case. The goal of `std::constant_wrapper` is to accept other `std::constant_wrapper`s in its operator

overloads, and to return `std::constant_wrappers` from them, so that computations are performed *entirely* within the type system.



On the contrary, `std::fn` would simply invoke the callable type it wraps, forwarding arguments to it, and return the result of that invocation.

Furthermore, C++29 `std::fn` with a call operator could provide the same functionality to `std::function`, `std::move_only_function`, and `std::copyable_function`; it would simply be invocable like function pointers or lambdas. Even without any constructors added to the standard wording, implementations are free to apply small-object optimization, i.e. no allocations are necessary, which may improve quality of implementation further. This optimization is simply part of the "as-if rule".

A `std::fn` with a call operator *just works* with all function wrappers and algorithms, exactly as expected, and with the same semantics as it has in `std::function_ref`.

§ 2.3. Making `std::nontype` useful for producing function pointers

Since the C++29 `std::fn` would be convertible to a function pointer, it could also be used for producing and converting function pointers. `std::fn` would be convertible to a function pointer type, possibly with conversions of parameters and the returned value being applied, not just if there is an "exact match" in signature.



EXAMPLE: In the following code, `is_prime` is wrapped in `std::fn`, which is convertible to `bool (*)(int)`, not just `bool (*)(long)`, because `int` is convertible to `long long`.

```
bool is_prime(long long x);  
bool (*)(int) = std::fn<&is_prime>;
```

This may be useful for passing function pointers to C APIs when on the C++ side, there is a slight mismatch between the C++ function signature and the expected function pointer type.

Similarly, C++29 `std::fn` could be used to produce function pointers to arbitrary callable types such as lambdas, `structs` with a call operator, etc. All of this is feasible by creating a function "on the fly" within a conversion operator template. See [§4.2. Possible implementation of `std::fn` for C++29](#) for specifics.

§ 2.4. Naming

While using such a short and valuable name as `std::fn` would make little sense if `std::nontype` was simply being renamed with no further plans, we have *big* plans. `std::fn` would be *the* idiomatic way of lifting function pointers into the type system.

`std::fn` is inspired by the `std::not_fn` and `std::mem_fn` function templates. The future C++29 `std::fn` would have similar purpose to `std::not_fn`; it just wouldn't negate the result of an invocation.

However, such a short and valuable name is likely controversial. A plausible alternative is to use `std::fw` and `std::function_wrapper`, analogous to `std::cw` and `std::constant_wrapper`, or `std::const_fn` and `std::const_fn_t`, analogous to `std::not_fn`. 



NOTE: Some authors initially had concerns about `fn` possibly being used as a namespace for functional programming in the future. However, the standard library generally has verbose namespaces (e.g. `filesystem`), and suggestion to add shorthands (e.g. `fs`) were historically rejected.

§ 2.5. Change of header

Consider that `std::fn` is a more "functional name" than `std::nontype`, and that the goal is to follow up the rename with a call operator added in C++29, `<functional>` is a more appropriate header than `<utility>`.

Since `std::function_ref` is located in `<functional>` anyway, this change is largely inconsequential. It just means that users wouldn't be able to obtain `std::fn` through `<utility>` on its own, but we don't see strong motivation to do so.

§ 2.6. Interoperability with third-party `function_refs`

It is worth noting that the proposed C++29 `std::fn` would work out-of-the-box with third-party `function_ref` implementations such as `llvm::function_ref`. Notably, `std::fn` would be a variable template, meaning objects of type `std::fn_t` have static storage duration. Therefore, binding an `llvm::function_ref` to `std::fn` does not risk creating a dangling reference.

However, it is still possible to produce a dangling reference to `std::fn` by deliberately copying it and creating a temporary object:

```
void f();
llvm::function_ref<void()> x = std::fn<f>;           // OK, safe
llvm::function_ref<void()> y = auto(std::fn<f>);    // dangling reference
```

This is also one of the reasons why `std::function_ref` should special-case `std::fn_t` in its set of constructors. Other function wrappers need no special cases because they take ownership over the wrapped callable anyway.

§ 3. Alternatives considered

Besides the proposed approach, there are other possible solutions. However, the author position is that every one of them is worse than what is proposed.

§ 3.1. Using a `std::stateless` constructor tag instead

[P3740R0] proposed to replace `std::nontype` with a set of constructors that take a `std::stateless` constructor tag. This idea essentially died the moment LEWG saw the Tony Table

which compares how users would pass function pointers to `std::function_ref` with `std::stateless` tags compared to `std::nontype`.



This alternative was not polled, but it was obvious during discussion that this option would not achieve consensus. See [\[P3740R0\]](#) for details.

§ 3.2. Removing `std::nontype` with no replacement, revisiting for C++29

Since there is a lot of active work in this area, perhaps we could simply shove `std::nontype` into C++29 and deal with the problem later. As demonstrated in [§1.2. Refresher on what `std::nontype` accomplishes](#), `std::nontype` covers crucial use cases such as

- constructing `std::function_ref` from a free function with no overhead, or
- constructing `std::function_ref` from an existing function pointer and `void*`, in the style of `qsort`.

This functionality is important to `std::function_ref`, and it has shipped in C++26 already. None of the authors see it as reasonable to rip this functionality out entirely for C++26.

§ 3.3. Waiting for `constexpr` function parameters

`std::integral_constant`, `std::constant_wrapper`, and `std::nontype` are – to an extent – temporary hacks. If we were able to write

```
function_ref constexpr F*);
```

... then the workaround of `std::nontype` would be obsolete.

At Kona 2023, EWG showed enthusiasm for this option when discussing [\[P1045R1\]](#) “`constexpr` Function parameters”.

”

Poll: P2781R3 “`std::constexpr_v`” and P1045R1 “`constexpr` function parameters” EWG would like to solve the problem solved by `std::constexpr_v` in the language, for example as proposed by P1045R1 or with “expression aliases”, rather than solving it in library. An implementation is desired. C++26 seems ambitious.

SF	F	N	A	SA
6	8	5	1	0

However, [\[P1045R1\]](#) has been abandoned by the original author, but picked up by a new author (proposal pending). Nonetheless, `constexpr` function parameters do not entirely obsolete our design, it is unclear if they will be in C++29, and the `std::nontype` functionality has already shipped in C++26. Therefore, it makes very little sense to wait for this feature.

§ 3.4. Replacing `std::nontype` with `std::constant_wrapper`

The seemingly obvious solution is to use `std::constant_wrapper` from [P2781R9]. The option to use `std::constant_wrapper` as a replacement for `std::nontype` was discussed during an LEWG telecon 2025-03-11.

In short, the problems stem from the fact that `std::constant_wrapper` is already invocable; it already has a call operator, whose semantics would be ignored by `std::function_ref`, in a way that is inconsistent with how `std::constant_wrapper` behaves anywhere else.

More details can be found in [P3740R1], as well as in [P3792R0] "Why `constant_wrapper` is not a usable replacement for `nontype`".

In any case, LEWG already decided (with paper-thin consensus) not to pursue this path.

§ 4. Implementation experience

§ 4.1. Simple rename for C++26

Renaming `std::nontype` requires no implementation experience because it is merely changing the name of a symbol; it obviously works. This is what we are proposing for C++26.

§ 4.2. Possible implementation of `std::fn` for C++29

An implementation for illustration purposes of the full C++29 `std::fn` may look as follows:

```
template<auto f>
struct fn_t
{
    using type = decltype(f);
    static constexpr bool is_function_ptr
        = std::is_function_v<std::remove_pointer_t<type>>;

    template<bool Noex, typename Ret, typename... Args>
    using func_type = Ret (*)(Args...) noexcept(Noex);

    constexpr
    operator type() const noexcept
        requires is_function_ptr
    { return f; }

    template<bool Noex, typename Ret, typename... Args>
        requires (Noex ? std::is_nothrow_invocable_v<Ret, type const&, Args...>
            : std::is_invocable_r_v<Ret, type const&, Args...>)
    constexpr
    operator func_type<Noex, Ret, Args...>() const
    {
        if constexpr (is_function_ptr && std::is_convertible_v<type, func_type<Noex, Ret, Args...>>)
            return f;
        else
            return [](Args... args) noexcept(Noex) -> Ret
            { return std::invoke(f, std::forward<Args>(args)...); };
    }
};
```



```

}

template<typename... Args>
    requires (!is_function_ptr)
static constexpr
std::invoke_result_t<type const&, Args...>
operator()(Args&&... args)
noexcept(std::is_nothrow_invocable_v<type const&, Args...>)
{ return std::invoke(f, std::forward<Args>(args)...); }
};

template<auto f>
constexpr fn_t<f> fn;

```

Note that the wording strategy for `std::fn_t` in C++29 would leave its details largely unspecified, which would allow for not implementing `operator()` at all, and for making `std::fn_t` invocable via surrogate function call (implicit conversion to function pointer, then calling that pointer). See § [Appendix — Follow-up wording for C++29](#)

§ 5. Wording

The following changes are relative to [\[N5008\]](#).

In [\[version.syn\]](#), update the feature-test macro:



```
#define __cpp_lib_function_ref 202306L 20XXXXL // also in <functional>
```

In [\[utility.syn\]](#), delete the declarations of `std::nontype` and `std::nontype_t`:



```

// nontype argument tag
template<auto V>
    struct nontype_t {
        explicit nontype_t() = default;
    };
template<auto V> constexpr nontype_t<V> nontype{};

```

In [\[functional.syn\]](#), change the synopsis as follows:



```

namespace std {
    [...]

    // [func.identity], identity
    struct identity;
    // freesta

    // constant function wrapper
    template<auto f>
        struct fn_t {
            explicit fn_t() = default;
        };
    template<auto f> constexpr fn_t<f> fn;

```

```
// [func.not.fn], function template not_fn
template<class F> constexpr unspecified not_fn(F&& f); // freesta
template<auto f> constexpr unspecified not_fn() noexcept; // freesta

[...]
}
```

In [func.wrap.ref.class], replace every occurrence of `nontype_t` with `fn_t`:



```
namespace std {
    template<class R, class... ArgTypes>
    class function_ref<R(ArgTypes...) cv noexcept(noex)> {
    public:
        // [func.wrap.ref.ctor], constructors and assignment operators
        template<class F> function_ref(F*) noexcept;
        template<class F> constexpr function_ref(F&&) noexcept;
        template<auto f> constexpr function_ref(nontype_t fn_t<f>) noexcept;
        template<auto f, class U> constexpr function_ref(nontype_t fn_t<f>, U&&)
        template<auto f, class T> constexpr function_ref(nontype_t fn_t<f>, cv T

        constexpr function_ref(const function_ref&) noexcept = default;
        constexpr function_ref& operator=(const function_ref&) noexcept = default;
        template<class T> function_ref& operator=(T) = delete;

        // [func.wrap.ref.inv], invocation
        R operator()(ArgTypes...) const noexcept(noex);

    private:
        template<class... T>
            static constexpr bool is-invokable-using = see below; // exposition o

        R (*thunk_ptr)(BoundEntityType, Args&&...) noexcept(noex); // exposition o
        BoundEntityType bound-entity; // exposition o
    };

    // [func.wrap.ref.deduct], deduction guides
    template<class F>
        function_ref(F*) -> function_ref<F>;
    template<auto f>
        function_ref(nontype_t fn_t<f>) -> function_ref<see below>;
    template<auto f, class T>
        function_ref(nontype_t fn_t<f>, T&&) -> function_ref<see below>;
}
```

In [func.wrap.ref.ctor], replace every occurrence of `nontype_t` with `fn_t`:



```
[...]

template<auto f> constexpr function_ref(nontype_t fn_t<f>) noexcept;

[...]

template<auto f, class U>
    constexpr function_ref(nontype_t fn_t<f>, U&& obj) noexcept;
```



```
[...]
template<auto f, class T>
constexpr function_ref(nontype_t fn_t<f>, cv T* obj) noexcept;

[...]

template<class T> function_ref& operator=(T) = delete;
```

21 Constraints:

- T is not the same type as function_ref,
- is_pointer_v<T> is false, and
- T is not a specialization of nontype_t fn_t.

In [func.wrap.ref.deduct], replace every occurrence of nontype_t with fn_t:



```
[...]
template<auto f>
function_ref(nontype_t fn_t<f>) -> function_ref<see below>;

[...]

template<auto f, class T>
function_ref(nontype_t fn_t<f>, T&&) -> function_ref<see below>;

[...]
```

§ 6. References

[N5008] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>

[P0792R14] *Vittorio Romeo, Zhihao Yuan, Jarrad Waterloo*. function_ref: a type-erased callable reference (2022-02-08)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0792r14.html>

[P1045R1] *David Stone*. constexpr Function Parameters (2019-09-27)
<https://open-std.org/JTC1/SC22/WG21/docs/papers/2019/p1045r1.html>

[P2472R3] *Jarrad J. Waterloo, Zhihao Yuan*. make function_ref more functional (2022-05-12)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2472r3.html>

[P2781R9] *Hana Dusíková, Matthias Kretz, Zach Laine*. std::constant_wrapper (2025-06-17)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2781r9.html>

[P2841R1] *Corentin Jabot*. Concept and variable-template template-parameters (2023-10-14)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2841r1.pdf>

[P3312R1] *Bengt Gustafsson*. Overload Set Types (2025-04-16)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3312r1.pdf>

[P3740R0] Jan Schultke. Last chance to fix std::nontype (2025-06-14)
<https://isocpp.org/files/papers/P3740R0.html>



[P3740R1] Jan Schultke. Last chance to fix std::nontype (2025-06-20)
<https://isocpp.org/files/papers/P3740R1.html>

[P3792R0] Bronek Kozicki. Why constant_wrapper is not a usable replacement for nontype
<https://wg21.link/P3792R0>

[GitHub1] Bronek Kozicki. Reference implementation of function wrappers with
std::constant_wrapper (2025-06-19)
https://github.com/MFHava/P2548/compare/master..Bronek:P2548:bronek/with_constant_wrapper

[GitHub2] Jan Schultke. Reference implementation of std::function_ref with std::stateless (2025-06-14)
https://github.com/Eisenwave/nontype_functional

§ Appendix — Follow-up wording for C++29

The following changes are relative to [N5008], with the changes in §5. Wording applied.

In [version.syn], create a feature-test macro:



```
#define __cpp_lib_fn 20XXXXL // also in <functional>
```

In [functional.syn], change the declaration of fn and fn_t as follows:



```
// [func.fn], constant function wrapper
template<auto f>
struct fn_t {
explicit fn_t() = default;
};
template<auto f> constexpr fn_t<f> fn;
```

Between [func.identity] and [func.not.fn], insert a new subclause:



Constant function wrapper

[func.fn]

```
template<auto f>
struct fn_t {
    explicit fn_t() = default;

    see below
};
```

1 Let ft be an object of type FT that is a (possibly const) specialization of fn_t, and let cf be a template parameter object ([temp.param]) corresponding to the constant template argument of FT. Then:

- FT is a trivially copyable type, such that FT models semiregular and is_empty_v<FT> is true;

- `ft` is a simple call wrapper (`[func.require]`) with no state entities and with the call pattern `invoke(cf, call_args...)`, where `call_args` is an argument pack used in a function call expression (`[expr.call]`), except that any parameter of the function selected by overload resolution may be initialized from the corresponding element of `call_args` if that element is a prvalue;
- for any type `R` and pack of types `Args`, both `cf` and `std::move(ft)` are convertible to:
 - `R(*) (Args...) if is_invocable_r_v<R, decltype(cf), Args...> is true;`
 - `R(*) (Args...) noexcept if is_nothrow_invocable_r_v<R, decltype(cf), Args...> is true.`

[Example:

```
bool is_even(long);  
bool(*ptr)(int) = std::fn<&is_even>;  
— end example]
```